Princess Sumaya University for Technology
جامعة الأميرة سمية للتكنولوجيا

Computer Science Department

Midterm Exam

CS-11313: Algorithm Analysis and Design
Spring Semester 2021/2022

Wed, 27/04/2022 @ 4:15 PM

Student Name:	**Student Number:**
Instructor:	**Signature:**

Instructions

- Time allowed: **60 Minutes**
- This booklet should be **7** pages long, including this page. If not, report this to the instructor.
- Answer all questions in the examination paper provided.
- Write your answers clearly: you may not receive full credit if your answer is unreadable.

Question	Max Mark	Your Mark
Q1	20	
Q2	20	
Q3	40	
Q4	20	
Total	100	

Question 1: Asymptotic Complexity [20 points]

(a) [4 pts] Define

1. $f(n) = \Omega(g(n))$:

There exist positive constants c & n. such that:

f(n) >= c.g(n) , for all n>=n.

or:

$$0 < \lim_{n\to\infty} \frac{f(n)}{g(n)} \leq \infty$$

2. $f(n) = O(g(n))$:

There exist positive constants c & n. such that:
f(n) <= c.g(n) , for all n>=n.

or:

$$0 \leq \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

(b) [4 pts] using the basic definition of BigO, Prove (or disprove) that $f = O(g)$

$$f(n) = 10n^2 + 2n + 1 \qquad g(n) = n^2$$

f(n) <= c.g(n) , for all n>=n.

$10n^2 + 2n + 1$ <= c. n^2, for all n>=n.
$10n^2 + 2n + 1$ <= 10. $n^2 + 2.n^2 + n^2$, for all n>=1
$10n^2 + 2n + 1$ <= 13. n^2, for all n>=1

c=13, n.=1

(c) [8 pts] For each pair of functions below:
1) Circle around the function that grows **asymptotically** faster (if any).
2) Give all the θ, O, and Ω relations that apply. You only need to mention the relations in which *f*(n) appears on the left-hand side.

- $f(n) = \sqrt{n}$ $\quad$ $g(n) = \sqrt{2}^{\log n}$ $\quad$ $f(n) = O(g(n)), \Omega(g(n)), \theta(g(n))$
- $\mathbf{f(n)} = n2^n$ $\quad$ $g(n) = 2^n$ $\quad$ $f(n) = \Omega(g(n))$……
- $f(n) = \log n!$ $\quad$ $\mathbf{g(n)} = n(\log n)^2$ $\quad$ $f(n) = O(g(n))$……
- $f(n) = (\log n)^{\log n}$ $\quad$ $g(n) = n^{\log\log n}$ $\quad$ $f(n) = O(g(n)), \Omega(g(n)), \theta(g(n))$

(e) [4 pts] If we know that the running time T(n) of some algorithm satisfies the following relations: T(n) = ω($n$log$n$) and T(n) = o(n^2). **List 2 functions that apply.**

1. – $n(\log n)^2$
2. – $n^{1.5}\log n$

Question 2: Recurrence Equations [20 points]

(b) [4 pts] Solve the following recurrence using the iterative method.

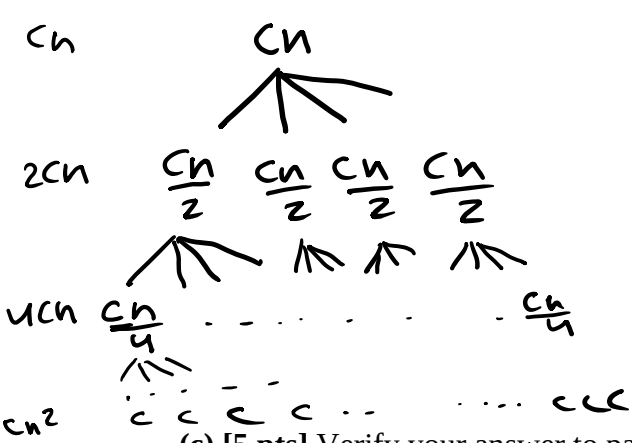
$$T(n) = \begin{cases} 3T(n-1) & if\ n > 0 \\ 1 & if\ n = 0 \end{cases}$$

T(n)=3T(n-1)
T(n)=9T(n-2)
T(n)=27T(n-3)
After k iterations
T(n)=3^kT(n-k)

We stop when T(n-k)=T(0), n-k=0, k=n

T(n)= 3^nT(0)= 3^n

(b) [8 pts] Solve the recurrence **T(n)= 4T(n/2) + cn** , **T(1) = c** using the ***recursion-tree*** method.



$$\sum_{i=0}^{\log n} 2^i cn = cn \sum_{i=0}^{\log n} 2^i = cn(2^{\log n+1} - 1)$$

$$= cn(2n-1) = \Theta(n^2) \quad \text{or}$$

$$cn^2 + \sum_{i=0}^{\log n - 1} 2^i cn = cn^2 + cn(2^{\log n} - 1)$$

$$= cn^2 + cn(n-1) = \Theta(n^2)$$

(c) [5 pts] Verify your answer to part (b) using the master theorem. If the master theorem can't be applied, state why.

L=n $^{\log_2 4}$ =n^2
F(n)=cn

$Cn=O(n^{2-E})$, $0<E<=1$

Case 1, $T(n)= \theta (n^2)$

(d) [3 pts] Express the running time **T(n)** of the following algorithm as a recurrence (do not write the base case).

```
int F(int n) {
   if (n <2) return 1; else c=1;
   for (int i = 0; i < 8; i++)
              F(n/8);
   for (int i=0;i<n^3;i++)
        c++;
```

T(n) = $8T(n/8)+n^3$

Question 2: Sorting and Selection Algorithms [40 points]

(a) [15 pts] Consider the following implementation of quicksort algorithm

```
void QuickSort (int A[], int low, int high){
     if (low<high){
             int pivotPoint = partition(A, low, high);
             QuickSort(A, low, pivotPoint-1);
             QuickSort(A, pivotPoint+1, high);
}}
 int partition(int A[], int low, int  high){
     int pivot=A[low];int i=low; int j=high+1;
     while (true) {
          do {i=i+1;} while ((i < high)&&(A[i]<pivot)) ;
          do {j=j-1;} while ((j > low )&&(A[j]>pivot)) ;
          if (i >= j) break;
          else{
              swap(A[i],A[j]);
              cout<<i<<"-"<<j<<endl; ←←←
          }
     }
     swap(A[low],A[j]);
     cout<<low<<"-"<<j<<endl;←←←
     return j;}
```

- **(6 points)** For the array (A) shown below, write the output printed by **QuickSort (A, 0, 3).**

9	6	7	5

0-3
0-0
1-1

- **(3 points)** If line 11 became `do {i=i+1;} while (A[i]<pivot);` Will the algorithm work correctly in all instances? If not, specify an instance that will cause an error.

  NO, 10 9 8 7 6

- **(3 points)** Modify the above partition algorithm such that it chooses a pivot randomly. Write only the modified lines.
  Before line 8

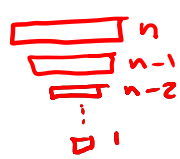
```
int pivotIndex = rand()%(high-low+1)+low;
swap(A[pivotIndex], A[low]);
```

- **(3 points)** Show that the worst-case running time for quicksort is $\Theta(n^2)$

If the partitioning method divides the array into two partitions, one empty and one of size n-1, and this situation is repeated throughout all recursive calls, the running time is T(n)=T(n-1)+cn, resulting in quadratic running time.
For example, if the array is sorted, all items in all recursive calls will be larger than the pivot. As a result, the array will be divided into two partitions, the first is empty and the second is of size n-1.

T(n)=T(n-1)+cn , T(1)=c
T(n)= T(n-2) + c(n-1)+cn
T(n)= T(n-3) +c(n-2)+ c(n-1)+cn
After k iterations:
T(n)= T(n-k) +c((n- (k-1)) + (n-(k-2))……n)

or

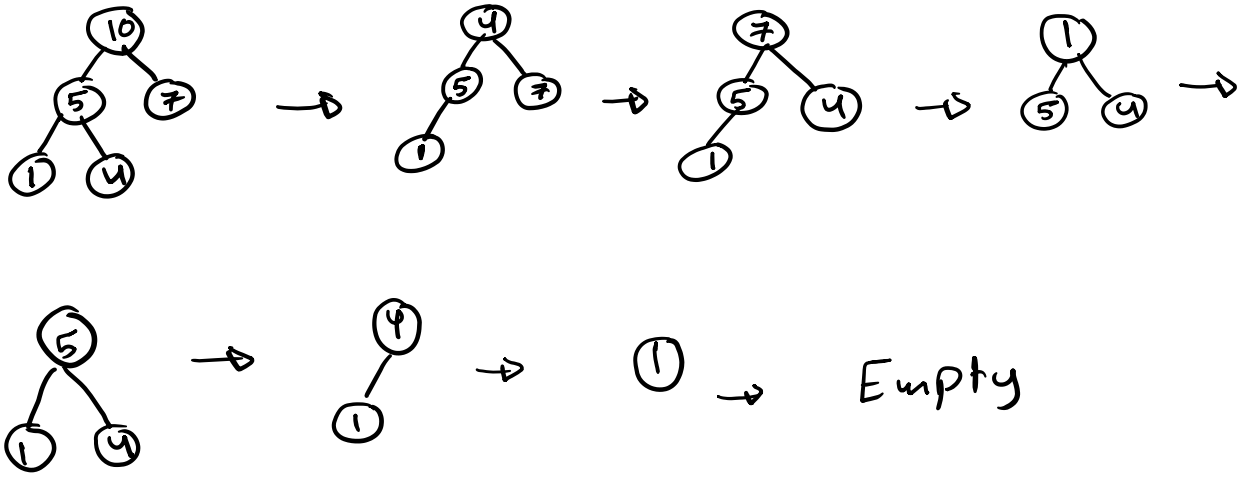


$\sum_{i=1}^{n} i = \Theta(n^2)$

Stop when n-k=1: k=n-1
T(n)= T(n-1-k) + c((n- (n-1-1)) + (n-(n-1-2))……n)= c+ c(2+3+…..n)= c+c $\sum_{i=2}^{n} i = c + c\left(\frac{n(n+1)}{2} - 3\right) = \Theta(n^2)$

(b) [10 pts] Given the following **max-heap** [10, 5, 7, 1, 4].

- Heapsort consists of two phases. Assume that the first phase (i.e., construct/build heap) is already executed. Show the execution of the second phase to sort the above *max*-heap. Draw the heap - as a tree- after each change. How many comparisons will the second phase make on the above array? How many swaps? Draw the heap - as a tree- after each change **(4 points**



Number of swaps: 6 Number of comparisons:5 **(2 points)**

- **(4 points)** Express the running time **T(n)** of the second phase in heapsort as a recurrence.

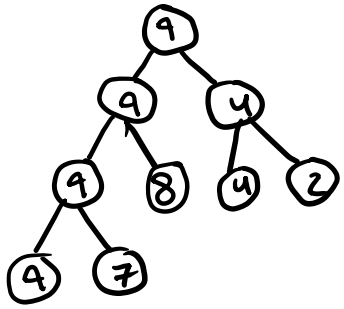
  T(n)=T(n-1)+clogn

(c) [10 pts] The tournament method for finding the second-largest key in an array finds the maximum element by constructing a heap of size 2*n*-1. Assuming the following list of size 5, [8, 4, 2, 9, 7].

- **(4 pts)** Fill the heap with the correct values

0	1	2	3	4	5	6	7	8
9	9	4	9	8	4	2	9	7

- **(3 pts)** Draw the above heap as a tree



- **(3 pts)** list the candidate elements for being the second-largest key after the *maximum element has been found*.
  4, 8, 7

(d) [5 pts] How do we find the number that appeared the maximum number of times in an array of size n? *Suggest an algorithm whose running time is better than* $\Theta(n^2)$.
For example: if the array contains the following elements:

5	2	1	5	1	5	2	3	5	7

Then the answer is 5 because it has the maximum number of occurrences. There are several algorithms that might be used

```
findMode(a, size)
    if size=1 return a[o]
    mergeSort(a, size)          → Θ(n log n)
    count=0, maxCount=0
    for i = 0 to size -2
        if   a[i]=a[i+1]
            count++
        else
            if count > maxCount
                mode=a[i]
                maxCount=count
            count=0
    if count > maxCount
        mode=a[size-1]
        maxCount=count
    return mode
// mode is the most frequently occurring value.
```

→ $\Theta(n)$

Question 4: Dynamic Programming Algorithms [20 points]

(a) [12 pts] Consider the **Knapsack** problem studied in class and the following instance of the problem:

Knapsack capacity = 8 Kg
Item 1: weight = 2 Kg, value = 4 jd
Item 2: weight = 6 Kg, value = 12 jd
Item 3: weight = 3 Kg, value = 9 jd

- **(8 pts)** Use the ***dynamic programming*** **(DP)** algorithm (the bottom-up approach) studied in class to find an optimal solution (maximum *value possible*) *to* the above 0-1 Knapsack problem. Show the DP table.

0	0	0	0	0	0	0	0	0	
0	0	4	4	4	4	4	4	4	✓
0	0	4	4	4	4	12	12	16	✓
0	0	4	9	9	13	13	13	16	✗

- **(2 pts) Construct** the optimal solution based on the table values. **Show** your work on the table.

Item1, item2

- **(2 pts)** Write the **recurrence relation** used in the above dynamic programming to solve the 0/1 knapsack problem

Knapsack(i,j)=

$$\begin{cases} 0, & i=0 \\ \max(v[i]+knapsack(i-1, j-w[i]),\ knapsack(i-1,j)), & i>0 \ \&\& \ w[i] < j \\ knapsack(i-1,j)), & i>0 \ \&\& \ w[i] > j \end{cases}$$

(b) [8 pts] Complete the missing part in the following code used to construct the optimal solution of 01-knapsack.

```
int i=n;int j=m;
while(i>0 && j>0) {
      if(K[i][j]==K[i-1][j]) //K is the DP table used in 01knapsack problem
            i= i-1
      else {
            cout<<"x"<<i<<" is included."<<endl;
```

```
            j=j-w[i]
            i=i-1
        }
```